

Technical Documentation File

Andrew Smith
Lewis Alexander
Preben Rasamanickam
Kyle Hannah

May 2026

1 Introduction

2 Hardware Architecture

2.1 System Overview

The hardware architecture is centred around an ESP32-S3 microcontroller, chosen for its dual-core processing capabilities, allowing task parallelisation between sensor polling and the main control loop. The sensor suite consists of four infrared (IR) sensors, a Time-of-Flight (ToF) distance sensor, and an MPU-6050 Inertial Measurement Unit (IMU). The MPU-6050 was selected due to its low cost, wide availability, and its inclusion of a 3-axis gyroscope, which helps with reducing APF-induced "ping-pong" oscillations using yaw damping. A dual H-bridge motor driver translates the control signals into physical movement.

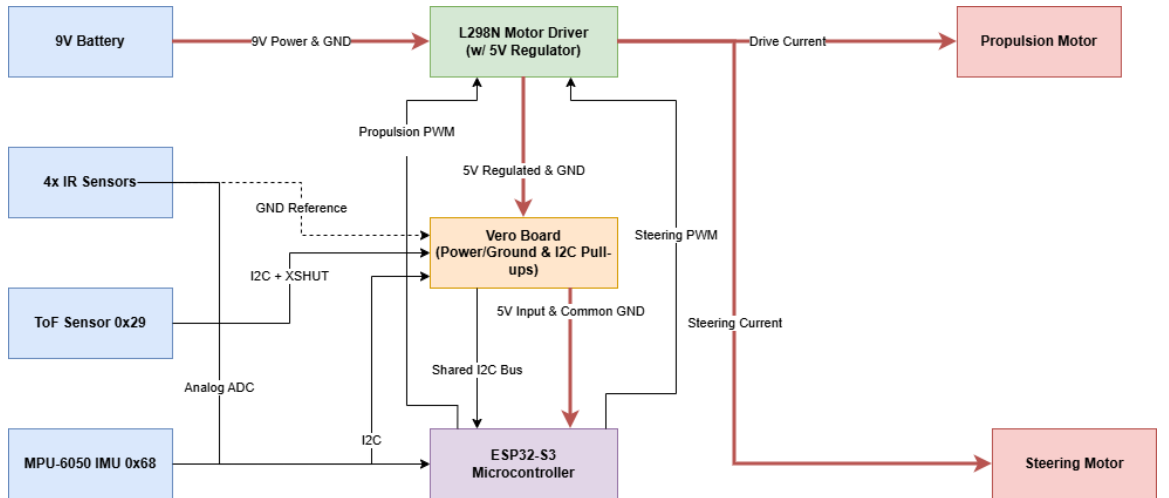


Figure 2.1: Overall hardware architecture showing the ESP32-S3, sensor suite, and motor driver interfaces

2.2 Component Mounting & Integration

Custom 3D-printed mounts were designed to secure the electronics and sensor suite to the chassis. The ESP32-S3 microcontroller and the 9V power

supply battery are housed in dedicated mounts positioned near the rear of the vehicle.

The ToF sensor is mounted securely at the front of the vehicle to detect forward obstacles. The four IR sensors are positioned at each corner (front-left, front-right, rear-left, and rear-right) using custom 3D-printed mounts attached to the tire rims. Because the vehicle's rear rims sit higher than the front ones, there is a height disparity between the front and rear sensors. This physical difference is compensated for by independently tuning the analogue thresholds for each sensor in the software. The IMU is mounted underneath the motor driver, near the vehicle's centre of gravity (CoG), to provide more accurate yaw rate feedback.

Insert 3D CAD models and physical placement diagrams of the ESP, battery, ToF, and IR mounts here

2.3 Wiring and Pin Allocation

The ESP32-S3 interfaces with the peripherals via ADC channels, I2C, and digital PWM pins. The pin allocation was carefully chosen to avoid conflicts, particularly avoiding the use of strapping pins or ADC channels that are active during boot for the motor PWM logic.

2.3.1 Motor Control Pins

The dual H-bridge motor driver requires four PWM signals to govern the forward/backward propulsion and left/right steering:

- Left Steering: GPIO12
- Right Steering: GPIO13
- Front Propulsion: GPIO14
- Back Propulsion: GPIO15

2.3.2 Infrared Sensors (ADC)

The analog outputs from the four IR sensors are routed to the ESP32 ADC channels:

- Front Right: ADC1_CH7
- Front Left: ADC1_CH6
- Rear Right: ADC1_CH5
- Rear Left: ADC1_CH4

2.3.3 I2C Bus (ToF & IMU)

To conserve GPIO pins, the ToF sensor (address 0x29) and the MPU-6050 IMU (address 0x68) share a single I2C bus:

- SCL: GPIO2
- SDA: GPIO1
- ToF XSHUT: GPIO9

2.4 Power Distribution and Custom Vero Board

A Vero board was utilised to manage power, ground, and the required I2C pull-ups. This approach provided the physical reliability needed for a moving chassis without the long lead times of a custom PCB.

- **Grounding:** The four IR sensors, ESP32, ToF, and IMU share a common ground. These are routed to the Vero board and tied to the motor driver ground to establish a consistent reference voltage across the entire system.
- **3.3V Power:** A dedicated distribution rail on the Vero board splits the 3.3V supply from the ESP32 to power the low-voltage sensor peripherals.
- **I2C Pull-ups:** The I2C communication lines (SCL and SDA) rely on pull-up resistors to maintain a logic high during idle states. These resistors are integrated directly into the Vero board circuit.

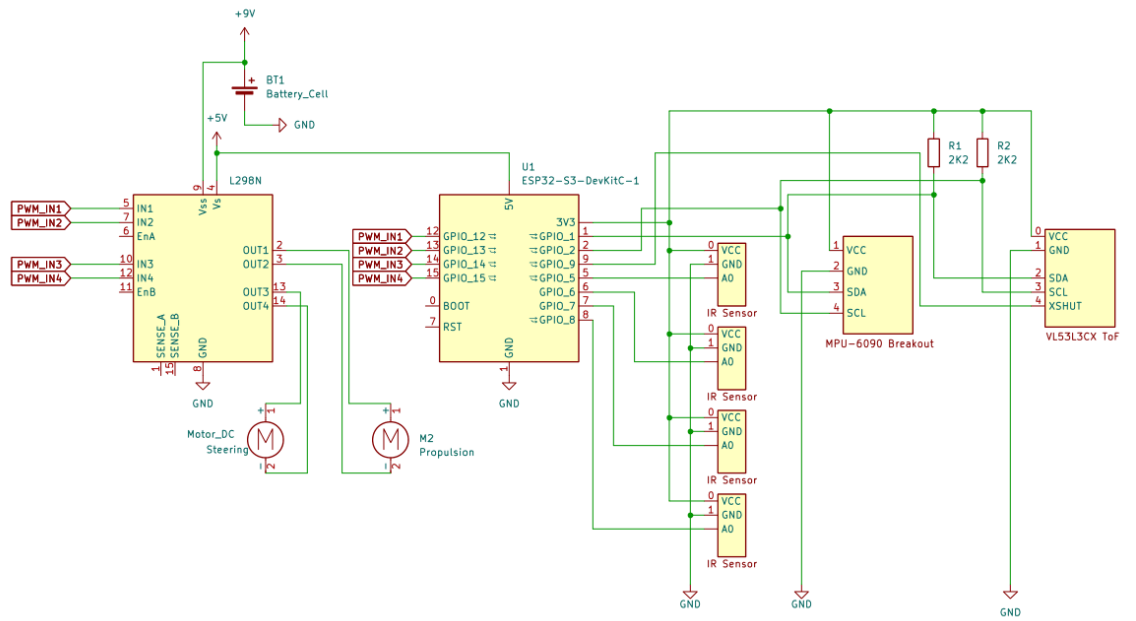


Figure 2.2: Electrical schematic of the final hardware wiring and signal distribution

3 Software Architecture & Finite State Machine

3.1 System Architecture Overview

The software architecture runs on FreeRTOS on the ESP32-S3 and keeps sensor collection, decision-making, and motor control separate. Although the ESP32-S3 has two cores, explicit core pinning was not needed; FreeRTOS SMP is used to spread the sensor tasks across the available cores. This keeps the path-planning code responsive and less affected by polling delays.

Data flows in one direction: Sensors → FSM → Navigation (APF) → Motor Control.

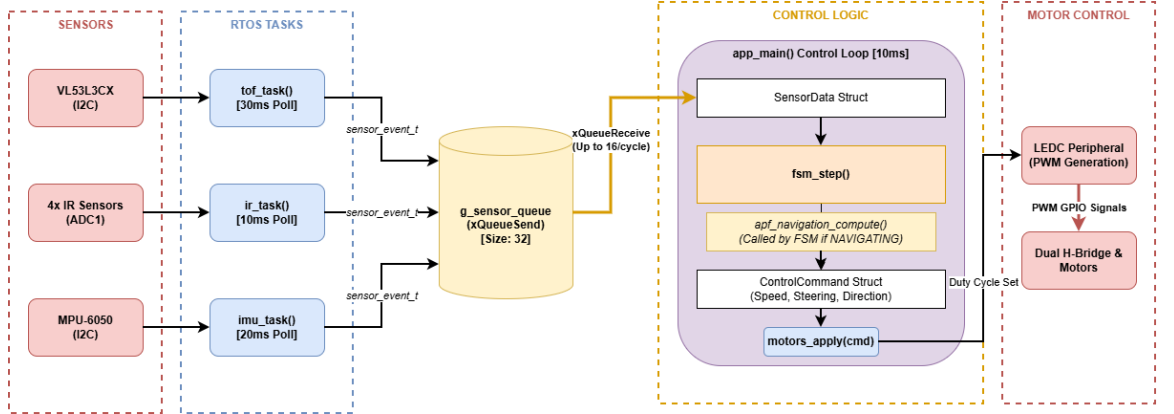


Figure 3.1: Overall software architecture showing the queue-based sensor pipeline, FSM, navigation, and motor control layers

3.2 Hardware Abstraction Interfaces

Data structures sit between the subsystems so the higher-level logic does not need to read hardware registers or work directly with raw PWM output.

3.2.1 SensorData Struct

The `SensorData` structure normalises all inputs before they reach the navigation logic. Infrared (IR) readings are calibrated onto a scale from 0.0

for white floor to 1.0 for black tape, and the Time-of-Flight (ToF) reading is converted into centimetres. This keeps the path-planning algorithm independent of the sensor hardware and mounting height.

3.2.2 ControlCommand Struct

Motor commands are standardised through a `ControlCommand` structure with three fields:

- **Speed:** A normalised float between 0.0 and 1.0.
- **Steering:** A normalised float between -1.0 (full left) and 1.0 (full right).
- **Direction:** An integer representing forward (+1) or reverse (-1).

Keeping navigation separate from raw PWM output makes the APF module independent of the L298N H-bridge, so hardware changes or motor retuning do not affect the algorithm.

3.3 RTOS Task Management & The 100Hz Control Loop

The ToF, IR, and IMU sensors each run in their own FreeRTOS task. Rather than overwriting global variables, which risks race conditions, each task packages its reading into a `sensor_event_t` packet and sends it to the thread-safe FreeRTOS queue (`g_sensor_queue`).

To stop fast-polling sensors from starving the FSM or tripping the watchdog, the central control task handles at most 16 events per cycle. It then uses `vTaskDelayUntil()` to maintain a 100Hz, or 10ms, control loop. That steady timing helps with smooth trajectory calculations and accurate IMU integration.

3.4 IMU Processing & Dynamic Bias Calibration

MEMS gyroscopes such as the MPU-6050 suffer from zero-bias drift and can report non-zero angular velocity even when the vehicle is still. If that error is left uncorrected, heading drift builds up over time.

The system updates the gyroscope bias whenever the vehicle is at rest. It treats the vehicle as stationary when the gyroscope magnitude stays below $12^\circ/s$ and the accelerometer reads roughly Earth's gravity ($1g \pm 0.2g$) for at least 0.5 seconds. During these periods, the zero-bias is updated with an Exponential Moving Average (EMA) filter:

$$\alpha = \frac{\Delta t}{\tau + \Delta t}$$

$$bias_{new} = (1 - \alpha) \cdot bias_{old} + \alpha \cdot \omega_{raw}$$

Where Δt is the polling interval and τ is the time constant. Once the vehicle starts moving, the bias update stops. The stored bias is then subtracted from the raw yaw rate (ω_{raw}) before the value reaches the FSM.

3.5 Finite State Machine (FSM)

The system is governed by a Finite State Machine configured through a central `SystemConfig` struct, which keeps the runtime thresholds and delays in one place.

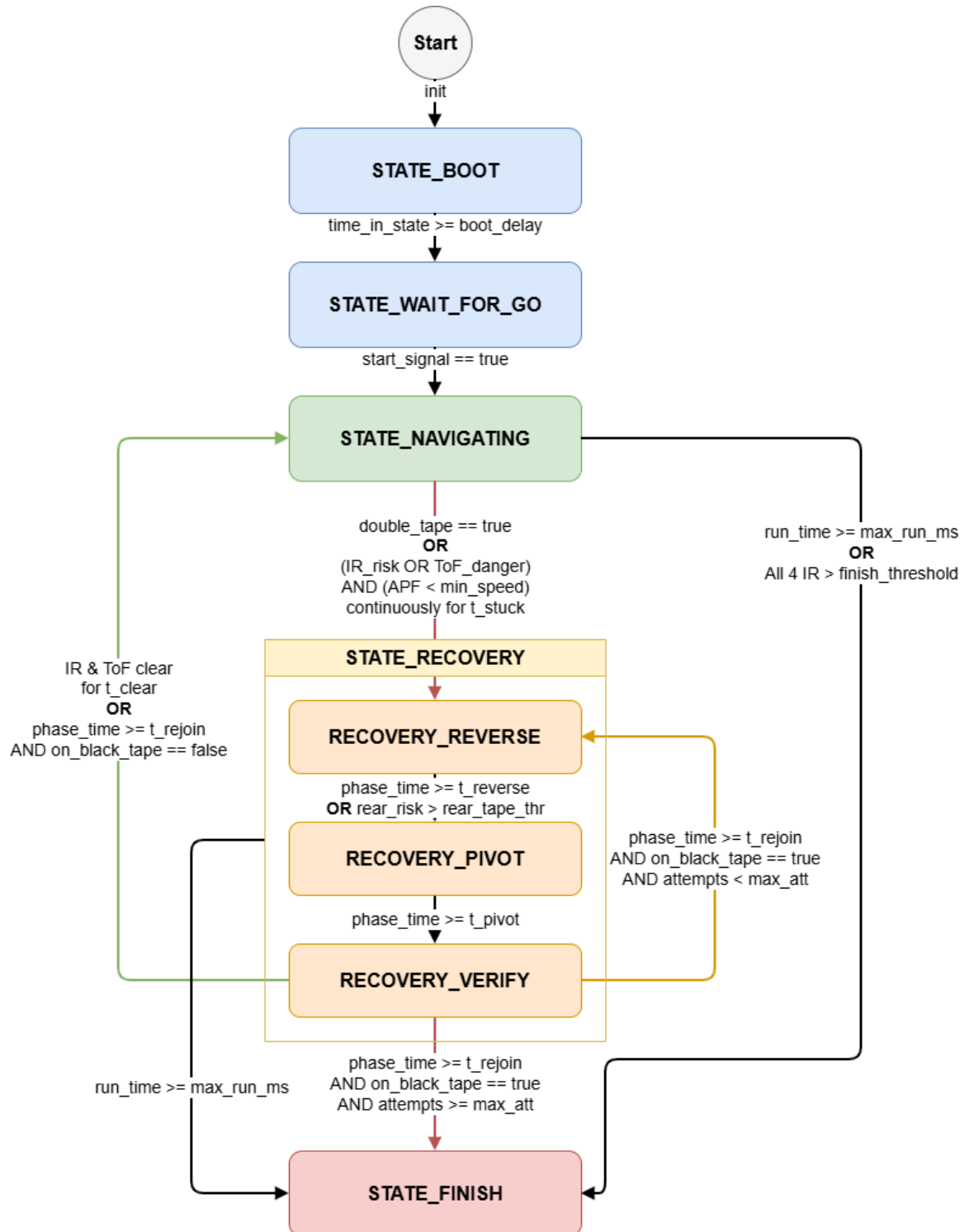


Figure 3.2: Finite state machine for boot, wait, navigation, recovery, and finish handling

- **STATE_BOOT:** A required 750ms delay immediately after power-on. This gives the ToF and IR sensors time to initialise before the system starts reading data.
- **STATE_WAIT_FOR_GO:** The system idles until the user presses the ESP32 BOOT button to start the run.
- **STATE_NAVIGATING:** The main operating state. Sensor data is fed into the Artificial Potential Field (APF) to produce the `ControlCommand`. The FSM also watches for the finish line, which is detected when all four IR sensors exceed the black-tape threshold at the same time.
- **STATE_RECOVERY:** An autonomous intervention state triggered when the vehicle becomes stuck.
- **STATE_FINISH:** Reached either by crossing the finish line or by exceeding the 2-minute run limit. All motor outputs are set to zero.

3.6 APF Local Minima Detection & Recovery Routing

A known limitation of Artificial Potential Fields is the local minima problem. For example, if the vehicle drives into a 90-degree corner, the repulsive forces from the black tape can cancel out and the APF may command zero target speed. The vehicle then becomes stuck. Strictly proportional APF implementations can also produce a ping-pong effect when the vehicle bounces between corridor walls because the control loop has no damping.

To reduce high-frequency lateral oscillations, the APF target heading (θ_{raw}) is damped using the real-time yaw rate (ω_{yaw}) from the IMU. An adjustable damping gain ($K_{damping}$) converts angular velocity into a steering penalty and smooths out large command spikes during rapid rotation:

$$\theta_{target} = \theta_{raw} - (\omega_{yaw} \cdot K_{damping})$$

To avoid getting stuck in local minima, the FSM checks whether the vehicle is in a high-risk zone, based on ToF and IR proximity, while the APF speed command stays below the minimum threshold (`min_apf_speed`, set to 0.03). If that condition lasts for the timeout period (`t_stuck_ms` = 500ms), the FSM transitions to **STATE_RECOVERY**. A double-tape panic override is also used: if both front IR sensors report high risk at the same time, the FSM skips the timeout and enters recovery immediately to keep the vehicle inside the field boundary.

The recovery sequence operates in three sub-phases:

1. **REVERSE:** The vehicle briefly stops to lose forward momentum, then reverses. A steering bias is applied based on which front IR sensor sees the highest risk, which helps pivot the rear wheels into free space. Reversing stops immediately if the rear IR sensors detect tape.

2. **PIVOT:** The vehicle shifts into forward gear and steers sharply away from the danger zone to break the APF's local minima equilibrium.
3. **VERIFY:** The vehicle resumes APF control under a strict speed limit. It must see clear floor, using safe IR and ToF readings, for a continuous period of `t_clear_ms` before returning to **STATE_NAVIGATING**. If tape is detected during verification, the recovery cycle repeats.

4 Navigation & Motor Control

4.1 Motor Controls

4.1.1 Overview

The motor control subsystem governs the movement of the vehicle and is implemented within a single software file. It receives input from the pathfinding algorithm and translates this into control signals for the two on-board motors. The original motors are retained, and no modifications to the chassis are required.

4.1.2 Hardware Architecture

The motor control subsystem consists of two DC motors: a rear-mounted drive motor responsible for propulsion and a front-mounted motor used for steering via wheel pivoting. This configuration is retained from the original chassis design, avoiding physical modifications and ensuring compliance with project requirements on chassis dimensions. Maintaining the existing layout also reduces mechanical complexity and ensures compatibility with the pre-defined mounting structure.

Motor control is achieved using an L298N Dual H-Bridge Motor Driver Module, which provides an interface between the ESP32 microcontroller and the motors. The driver enables bidirectional control of each motor using an H-bridge configuration, allowing both forward and reverse operation.

The ESP32 supplies control signals to the driver through four digital input pins, which determine the direction of rotation for each motor. These input pins are arranged in pairs, with IN1 and IN2 controlling the steering motor, and IN3 and IN4 controlling the drive motor. By setting one pin in each pair high while the other is low, the polarity of the voltage applied to each motor is controlled.

The mapping between ESP32 GPIO pins and motor driver inputs is shown in Figure 4.1. In the implemented control scheme, each GPIO pin is associated with a specific vehicle movement command. Before a new command is issued, all motor control pins are driven low, ensuring that only one control signal is active at a time. This simplifies direction control and prevents conflicting input states from being applied to the motor driver.

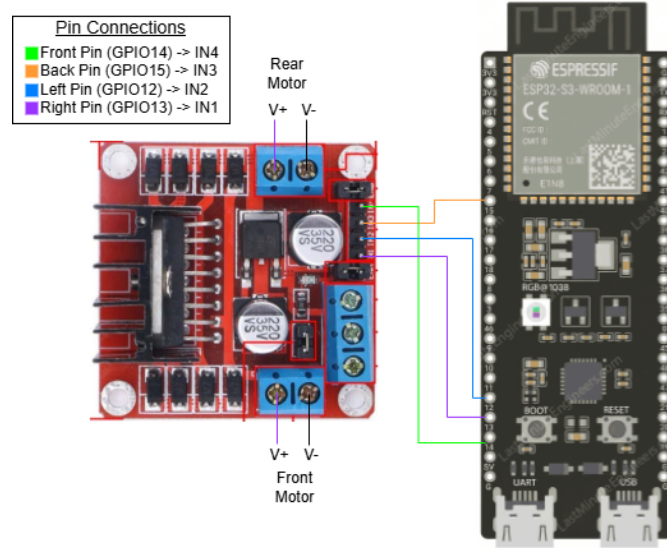


Figure 4.1: Mapping between ESP32 GPIO pins and motor driver inputs

Speed control is achieved using Pulse Width Modulation (PWM) signals generated by the ESP32 and applied to the enable pins of the driver. As the control signals are digital, the amplitude of the PWM signal remains constant, and motor speed is regulated by adjusting the duty cycle.

The L298N module is powered separately from the ESP32 logic supply, providing sufficient current to drive the motors. While the ESP32 operates at 3.3 V logic levels, the driver is compatible with these signals, enabling direct interfacing without additional level shifting circuitry.

4.1.3 Control Logic

The motor control system is implemented using four PWM channels, each corresponding to one input pin of the motor driver (IN1–IN4). At system initialisation, the `Motors_Init()` function is called, configuring all PWM signals with a duty cycle of 0%, ensuring that no motors are driven by default.

When the `Motors_Apply()` function is invoked, control inputs from the pathfinding module are used to update the duty cycles of the relevant PWM channels. The order of operations is illustrated in Figure 4.2.

A non-zero duty cycle is applied only to the PWM channel corresponding to the desired direction of motion, while the complementary channel in the input pair remains at 0%. This ensures that only one control signal is active for each motor at any time, preventing conflicting inputs from being applied to the motor driver.

Steering and propulsion are handled independently. The steering input

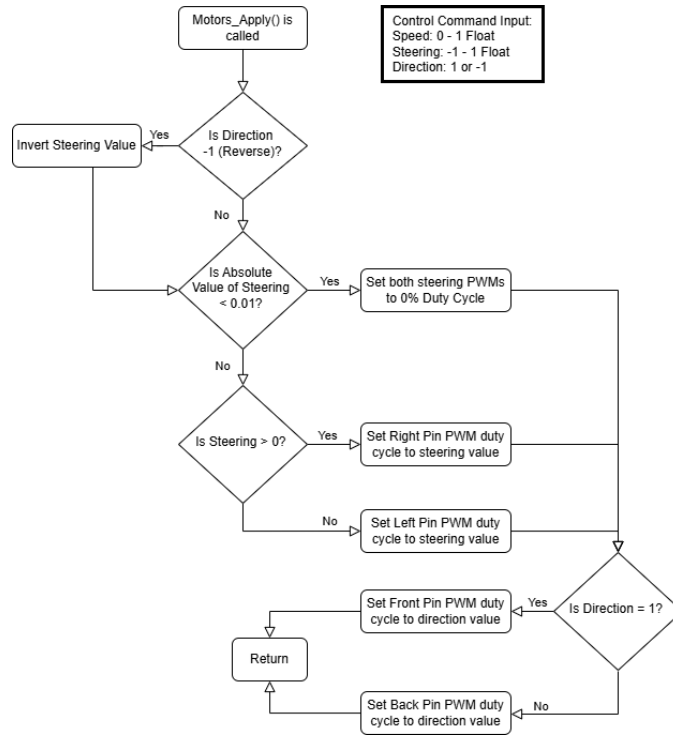


Figure 4.2: Motor control logic flow

determines which of the steering-related PWM channels is activated and to what duty cycle, while the direction input determines which propulsion-related channel is driven. This structure allows high-level motion commands to be translated directly into low-level motor control signals.

The control logic begins by evaluating the direction input. If the direction is negative (reverse), the sign of the steering value is inverted. This is required because the pathfinding algorithm defines steering relative to the forward-facing orientation of the vehicle; when reversing, the steering response must be inverted to achieve the intended motion.

Following this, the steering value is evaluated to configure the steering PWM outputs. To prevent unintended drift caused by very small floating-point values, a deadband is applied. If the magnitude of the steering input is below 0.01, both steering PWM channels are set to a duty cycle of 0%. If the value is positive, the vehicle steers right; if negative, it steers left.

Finally, propulsion is configured by evaluating the direction input and applying the speed value to the corresponding PWM channel. This step is performed after steering configuration to ensure that the wheels are correctly oriented before motion begins. This ordering reduces the likelihood of incorrect movement if an interrupt occurs during execution of the `Motors_Apply()` function.

4.1.4 Software Implementation

The motor control subsystem is structured as a modular interface consisting of three primary functions: `Motors_Init()`, `Motors_Apply()`, and `Motors_Stop()`. This structure provides a clear abstraction layer between the motor control logic and the higher-level finite state machine.

The `Motors_Init()` function is responsible for configuring the PWM channels used to drive the motor driver inputs. PWM generation and duty cycle control are implemented using the ESP32 LEDC library. All PWM channels are initialised with a duty cycle of 0%, ensuring that the motors remain inactive on system startup.

The `Motors_Apply()` function implements the control logic described in the previous section and serves as the main interface for motor control. It receives speed, steering, and direction inputs from the pathfinding module and translates these into PWM duty cycles applied to the motor driver inputs.

Input validation is performed within this function to ensure robust operation. Speed is constrained to $[0, 1]$, steering is constrained to $[-1, 1]$, and out-of-range values are clamped before applying PWM. The direction input is expected to be either -1 or 1; invalid values default to forward motion. This behaviour is intentionally chosen to prioritise forward displacement in the event of unexpected pathfinding outputs, maximising progress through the course while minimising reverse motion that does not contribute to overall advancement.

The function updates the PWM outputs by mapping validated input values directly to duty cycles, ensuring that only the channel corresponding to the intended direction of motion is active at any given time. This maintains consistency with the control scheme described previously and prevents conflicting motor commands.

The `Motors_Stop()` function provides a simple mechanism to halt all motor activity by setting the duty cycle of all PWM channels to 0%. This function is used both during testing and as a safety mechanism during operation.

4.1.5 Testing

The motor control subsystem was validated using a dedicated test driver, which repeatedly invoked the `Motors_Apply()` function with predefined input values. These inputs were selected to cover both standard operating conditions and edge cases near decision thresholds within the control logic.

Each test case was executed on the physical hardware for a duration of 2 seconds, followed by a 0.5 second pause. During this pause, the `Motors_Stop()` function was called, ensuring that both motion and stopping behaviour were verified. This approach allowed the response of the motors to be clearly ob-

served for each input condition.

Test cases included nominal inputs, such as full forward motion and directional steering, as well as edge cases involving small steering magnitudes and direction changes. Due to the physical nature of the system, results were validated through observation of motor behaviour, confirming that the vehicle responded in accordance with the expected direction and speed.

The test suite was structured into four categories: normal operation, boundary conditions on speed, boundary conditions on steering, and extreme input values. Normal tests verified expected behaviour for typical motion commands, including forward, reverse, and combined steering inputs. Boundary tests evaluated system behaviour at and beyond defined input limits (e.g. ± 1.0), ensuring stable operation near decision thresholds. Extreme tests used significantly out-of-range values to confirm that the system remained stable and did not produce undefined behaviour.

A representative subset of test cases is shown in Table 4.1, with the full test suite provided in Appendix A.

Test Name	Input Value	Expected Result	Actual Result
Forward Only	[0.9f, 0.0f, 1]	Car drives straight forward, 75% speed	Car drives straight forward, 75% speed
Left Only	[0.0f, -0.9f, 1]	Front wheels turn left at a medium rate	Front wheels turn left at a medium rate
Reverse Left	[0.0f, 0.9f, -1]	Car reverses and turns left at medium rate	Car reverses and turns left at medium rate
Forward Speed Edge Over	[1.01f, 0.0f, 1]	Car drives straight forward, full speed	Car drives straight forward, full speed
Forward & Left Under	[0.99f, -0.99f, 1]	Car drives forward, turning left, almost full speed	Car drives forward, turning left, almost full speed
Extreme Back & Right	[20.0f, -20.0f, -1]	Car reverses right at full speed	Car reverses right at full speed

Table 4.1: Motor control test results

4.2 Path Planning

4.2.1 Overview

Need to add stuff on when an obstacle is too close it starts reversing when thats implemented.

The path planning subsystem is responsible for converting sensor data into motion commands for the vehicle. Unlike a map-based planner, this implementation uses an artificial potential field (APF) approach, in which

the vehicle is guided by a combination of attractive and repulsive influences derived from the sensors. The resulting heading and speed are then passed into the motor control subsystem.

This method was chosen because it is compact, computationally efficient, and allows for a small non complete map to be used. The path planning logic operates directly on the inputs from the infrared and distance sensors along with the IMU data. By using continuous sensor feedback, the vehicle is able to react dynamically to changes in the environment while maintaining smooth motion.

4.2.2 Sensor Interpretation and Field Construction

The path planning algorithm uses four infrared (IR) sensors positioned around the vehicle, together with a front-facing time-of-flight (ToF) distance sensor and an inertial measurement unit (IMU). The IR readings are first clamped to prevent invalid values from influencing the navigation logic. These readings are then grouped into left, right, front, and rear loads, allowing the vehicle to estimate its location relative to nearby walls.

A measured wall heading is computed using the relationship:

$$\theta_{wall} = \arctan 2(L - R, F - B)$$

where L and R represent the combined left and right sensor loads, and F and B represent the combined front and rear loads. This heading is wrapped to the range of $[-180^\circ, 180^\circ]$ to ensure the path planning outputs are possible for the vehicle to achieve.

The IMU is used to steady the APF output rather than define the navigation field itself. The drift-compensated yaw rate from Chapter 3 is fed into the planner to soften sharp steering changes and reduce the oscillation that can occur when the APF repeatedly rebounds from walls and black tape. When `imu_heading_valid` is set, the IMU heading can also nudge the stored wall reference, but IR and ToF data still provide the main navigation input.

During initialisation, the subsystem stores an *original wall heading* as a stable reference direction. Once the ToF sensor reports a valid distance measurement, that heading is updated each cycle using the current IR balance and distance reading so the reference can adapt gradually to the course.

4.2.3 Control Logic

The navigation decision is based on the combination of attractive and repulsive fields. The attractive component encourages the vehicle to move in the direction of the stored wall heading, which acts as the intended course direction. Its magnitude is scaled according to the measured front distance,

meaning that the vehicle is encouraged to move more strongly when there is sufficient space ahead.

Repulsive forces are generated from the IR sensor imbalance. These forces oppose proximity to nearby tape or obstacles and are calculated using the difference between the left and right sensor loads, as well as the front and rear loads. The repulsive field is then combined with the attractive field to produce a resultant vector representing the desired direction of travel.

The final heading is calculated from this resultant vector using:

$$\theta_{desired} = \arctan 2(F_y, F_x)$$

where F_x and F_y are the horizontal and vertical components of the combined field. This approach allows the vehicle to balance course-following behaviour with obstacle avoidance in a single control step.

Speed is estimated from the magnitude of the resultant field and then adjusted according to environmental risk. Motion is reduced when the vehicle is close to a wall or obstacle, it is also reduced when the IR sensors indicate a strong tape danger signal. In addition, if the repulsive component becomes large, the maximum speed is capped to reduce oscillation and improve stability. This produces smoother behaviour in confined or ambiguous sections of the course.

To prevent abrupt changes in motion, the heading command is filtered using temporal smoothing by interpolating the new output with the previous heading. This reduces jitter and helps the vehicle maintain stable motion across successive control cycles.

4.2.4 Software Implementation

The path planning subsystem is implemented through two primary functions: `apf_navigation_init()` and `apf_navigation_compute()`. A third function, `apf_navigation_to_control()`, converts the resulting navigation command into the format required by the motor control subsystem.

The `apf_navigation_init()` function resets the internal state variables used by the planner. These include the stored wall heading, a validity flag indicating whether the reference heading has been established, and the previous heading and speed values used for smoothing. Initialising these values ensures that the first navigation cycle begins from a known state.

The `apf_navigation_compute()` function contains the main APF logic. It receives sensor data, validates and clamps the inputs, computes the attractive and repulsive components, and produces a desired heading and speed. Internal memory is updated at the end of each call so that the next iteration can make use of the previous output. This stateful design is important because it provides continuity between control cycles and improves navigation stability.

The `apf_navigation_to_control()` function maps the APF output into low-level drive commands. The heading is normalised to a steering range by dividing it by 40° and clamping the result to $[-1, 1]$. The speed value is also constrained to $[0, 1]$. The direction output is fixed to forward motion, meaning that the planner is designed primarily for continuous forward navigation rather than reversing manoeuvres.

This separation between navigation and motor control provides a clear abstraction layer. The path planner determines *where* the vehicle should move, while the motor control subsystem determines *how* that motion is physically applied to the motors.

4.2.5 Behavioural Characteristics

The implemented APF strategy provides several useful behaviours. The vehicle tends to follow a stable course when the surroundings are consistent, because the original wall heading acts as a persistent directional reference. When the environment changes, the repulsive field helps prevent collisions or tape violations by steering the vehicle away from local hazards. The smoothing stage then prevents the output from changing too abruptly, which is particularly beneficial when sensor readings fluctuate between adjacent control cycles.

The use of a distance-based speed reduction also improves safety. When the vehicle approaches a wall or obstacle, the planner automatically slows the vehicle down, giving the steering system more time to respond. Similarly, strong IR activity near the tape causes the vehicle to reduce speed, lowering the likelihood of overshooting the course boundary.

Overall, the path planning subsystem provides a lightweight and reactive navigation method that is well matched to the available sensors and embedded processing platform. Its continuous control structure allows the vehicle to maintain progress while dynamically adapting to local obstacles and course constraints.

5 Usage Instructions & Modifiability